

1.1 Model, design, and implementation

Statement-1: Model (M). It defines the relationship between files and timestamps, and the relationship between files and folders.

Statement-2: Design (D). It proposes a method for finding the latest archive given multiple archives and their timestamps.

Statement-3: Design (D). It proposes a way to track up-to-date archives using tables, but does not specify specific implementation details.

Statement-4: Implementation (I). It states the use of C language and the API provided by the operating system to implement the algorithm described in Statement-2.

Statement-5: Implementation (I). It uses an Excel spreadsheet to implement the tabular approach described in Statement-3.

Statement-6: Implementation (I). It describes an operating system that automatically maintains the tables described in Statement-3 and provides a button to allow users to easily find the latest files.

1.2 The pub/sub model

Question 1: In the publish/subscribe model, does a publisher need to know the identity of at least one of its subscriber? Why or why not?

In the publish/subscribe model, the publisher does not need to know the identities of any subscribers. This is because communication between publishers and subscribers occurs through a message broker. Publishers only need to publish messages to specific topics, while subscribers subscribe to topics of interest and receive related messages. The message broker is responsible for delivering messages from the publisher to all matching subscribers without the publisher knowing anything about the subscribers.

Question 2: For the following statement, if you think what it describes is true, write T; otherwise, write F and explain.

Statement: Within the scope of the publish/subscribe model, publishers can communicate directly with subscribers without the help of a message broker.

F

In the publish/subscribe model, communication between publishers and subscribers must go through a message broker. This is a basic principle of the model, which enables decoupling between publishers and subscribers, thereby improving the flexibility, scalability, and reliability of the system.

Time decoupling: Publishers and subscribers do not need to be online at the same time. Publishers can publish messages even if no subscribers are online at the moment, and subscribers can get the messages later.

Spatial decoupling: Publishers and subscribers do not need to know each other's existence or IP addresses. They only need to know the common topic (Topic), and then communicate through the proxy.

Synchronous decoupling: Publishers do not need to wait for subscribers to process messages, and vice versa. This contributes to system flexibility and scalability.

Question 3

Now it is your job to figure out a design using topics and messages to meet the above requirements. One thing to consider: how does the database know which topic to use when publishing a result to the specific user?

Using a global topic will not work because it will violate requirement (g).

In order to meet the above requirements, we can adopt the following design:

The user program generates a unique topic name, such as "topic1", and publishes the database access request to the public topic T along with the unique reply topic "topic1". The database subscribes to the public topic T and receives all request messages, parses the request message and obtains the unique reply subject "topic1" of the user program, then dynamically subscribes to the topic "topic1".

After the repository processes the request, it publishes the results to the topic "topic1", and the user program acts as a subscriber of the topic "topic1" and receives the reply message from the database.

1.3 The MQTT protocol

Question 1: For each of the following statement, if you think what it describes is true, write T; otherwise, write F and explain why.

Statement-1: In MQTT, the message is encapsulated in the payload of the control packet PUBLISH.

T

Statement-2: In MQTT, the messaging latency using QoS 1 will be approximately double of that of using QoS 0.

F

Although QoS 1 provides a higher level of message delivery guarantees than QoS 0, this does not mean that QoS 1 has twice the latency of QoS 0. Message delay will be affected by many factors, such as network conditions, message size, processing speed of the message agent, etc.

Question 2: Answer the following question in your own words and then compare it with the answer from ChatGPT (it is sufficient to use the free version):

Question: Study the QoS 2 semantics described in the MQTT v5 specification and in that explain in that context the meaning of packet identifier and the purpose of control packet PUBREL.

My answer:

1. Packet identifier: In the Variable Header part of the PUBREL packet, the first two bytes are the packet identifier. The same PUBREC packet also has a two-byte packet identifier. The function of the packet identifier in PUBREL and PUBREC is to ensure that the communication between the MQTT sender and the MQTT receiver is matching.
2. Purpose of PUBREL: In QoS level 2, when the MQTT receiver receives the message sent by the MQTT sender, the MQTT sender will send a PUBREC packet to the MQTT receiver to indicate that it has received the message. At this time, the MQTT sender will send a PUBREL packet in response to the PUBREC packet.

The answer from ChatGPT:

In MQTT v5.0, the PUBREL (Publish Release) control packet is part of the QoS 2 (Exactly Once) message delivery flow. It represents the second part of the four-step handshake required for QoS 2 communication. Here's an overview:

1. Packet Identifier: Each PUBREL packet includes a Packet Identifier, which is a 2-byte value used to uniquely identify the message. This identifier matches the one used in the previous PUBREC (Publish Received) packet, ensuring the message being released is the correct one. The packet identifier helps track the flow of messages and prevents the delivery of duplicate or incorrect messages.
2. Purpose of PUBREL: The PUBREL packet is sent by the publisher (or the receiver

of a QoS 2 message) after it has received a PUBREC packet. PUBREL acknowledges that the publisher is ready for the message to be released for final delivery. Upon receiving the PUBREL, the receiver responds with a PUBCOMP (Publish Complete) packet, completing the QoS 2 handshake. This ensures that the message is delivered exactly once, as required by QoS 2.

In summary, PUBREL is essential for the reliable delivery of messages when using QoS 2, allowing both the sender and receiver to confirm successful message processing and prevent duplicates.

Qualitative assessment:

I think my answer is more straightforward, clearly specifying each action and the objects involved in the action.

And the answer of ChatGPT is easier to understand and more suitable for explaining to ordinary people.